

MasterPy commands

Guida agli script Python® MS

Riferimento rapido per utenti MasterSap.

Sommario

Introduzione	2
Regole di base	2
Concetti fondamentali	2
Unità di misura	2
GitHub Repository	3
Intestazione dello script	3
Struttura tipica di uno script	4
Comandi MS	5
MS.SetMaterial	5
MS.Section	6
MS.Node	7
MS.DefineGroup	8
MS.ActivateGroup	9
MS.Beam	10
MS.Plate	11
MS.Macro	12
MS.SetUndo	13
MS.Redraw	13
Avviamento di uno script MasterSap nella console MasterPy	14
go	14
Esempio 1: edificio a due piani	15
Esempio 2: edificio multipiano	16
Esempio 3: volta a crociera (solo nodi)	19
Esempio 4: travatura reticolare Mohnie	21

Introduzione

MasterSap (abbreviato **MS**) è un programma di calcolo strutturale che supporta la modellazione tramite script Python®. Questi script permettono di generare automaticamente costruzioni tridimensionali composte da travi, colonne e piastre, riducendo i tempi di input e rendendo il modello facilmente parametrico e riproducibile.

Si utilizza l'ambiente MasterPy per il supporto di script in linguaggio Python® arricchiti con comandi specifici per MasterSap inseriti in una libreria detta **MS**.

Essa viene importata automaticamente dall'ambiente di esecuzione MasterPy che è una console.

Regole di base

Gli script devono contenere **solo caratteri ASCII**, i comandi sono 'case sensitive' quindi bisogna prestare attenzione ad inserire le lettere maiuscole e minuscole, cioè rispettare la sintassi dei comandi descritti in questa guida.

Per conoscere i fondamenti del linguaggio Python® si può fare riferimento al seguente link: www.python.org ed esplorare la documentazione disponibile.

Per avviare con successo uno script che venga quindi recepito in MasterSap o per inviare manualmente un comando MS, tramite l'ambiente a console MasterPy, il programma MasterSap deve essere in esecuzione con un progetto vuoto aperto.

Ogni script deve dichiarare le unità di misura e mantenere coerenza interna.

Quando si utilizzano variabili float usare il **punto** come separatore decimale (30.0, non 30,0).

Concetti fondamentali

Nodi: punti nello spazio 3D che collegano gli elementi strutturali. Ogni nodo ha un identificativo univoco (nodeID) e coordinate X, Y, Z.

Travi e Colonne: elementi monodimensionali che collegano due nodi. Le colonne sono travi disposte verticalmente. Ogni elemento ha un identificativo univoco (beamID).

Piastre: elementi bidimensionali triangolari o quadrilateri che collegano 3 o 4 nodi. Si usano per modellare pareti, platee, cupole e superfici curve. Ogni piastra ha un identificativo univoco (plateID).

Macro: elemento simile alla piastra ma di grandi dimensioni che MasterSap suddivide autonomamente in piastre più piccole.

Gruppi: contenitori logici per gli elementi (travi o piastre). Ogni elemento deve appartenere a un gruppo attivo. I gruppi servono di solito a identificare i piani di un edificio.

Unità di misura

Prima di generare qualsiasi elemento occorre stabilire l'unità di misura dello script: **cm** (centimetri) oppure **m** (metri). Tutte le coordinate, lunghezze e spessori devono essere espressi nell'unità scelta; se si propende per i comandi in cm e poi si inseriscono dati in m quei dati saranno interpretati in cm.

GitHub Repository

Il repository GitHub [MasterSap Python Scripts](#) è stato creato al fine di facilitare lo scambio di script e idee fra gli utenti.

Intestazione dello script

Per mantenere il repository ordinato e utile a tutti, ogni script deve avere un header iniziale con:

```
# =====  
# MasterSap Script  
# Titolo: <titolo breve>  
# Autore: <nome / nick>  
# Licenza: Apache-2.0  
# Testato su: MasterSap YYYY ReleaseNumber <es. MasterSap 2026 R1>  
# Unità: <kg | kN; cm | m>  
# Scopo: <cosa fa lo script in 1-3 righe>  
# Input: <parametri, file richiesti, assunzioni>  
# Output: <cosa crea/modifica in MasterSap>  
# Limitazioni: <vincoli noti, casi non gestiti, performance, ecc.>  
# =====
```

Questo stesso documento è contenuto nel repository.

Struttura tipica di uno script

Uno script MasterSap completo segue tipicamente questo schema:

```
# =====
# MasterSap Script
# Titolo: Test iniziale
# Autore: AMV
# Licenza: Apache-2.0
# Testato su: MasterSap 2026 R1
# Unità: <kg; cm>
# Scopo: Crea una trave in c.a. 30x50
# Input: no
# Output: 2 nodi e una trave
# Limitazioni: no
# =====

import MS

# 1. Imposta il punto di undo
MS.SetUndo()

# 2. Definizione dei materiali
matID = MS.SetMaterial('C25/30')

# 3. Definizione delle sezioni
MS.Section(1, 2, 'Rp', 30.0, 50.0)

# 4. Definizione dei nodi
n1 = MS.Node(0, 0.0, 0.0, 0.0)
n2 = MS.Node(0, 500.0, 0.0, 0.0)

# 5. Creazione gruppi ed elementi
MS.DefineGroup(1, 'TRAVE', 'Travi piano terra')
MS.ActivateGroup(1, 'TRAVE')
MS.Beam(0, matID, 1, n1, n2)

# 6. Ridisegno finale
MS.Redraw()
```

Comandi MS

Di seguito sono descritti tutti i comandi MS specifici di MasterSap. Si assume che il modulo MS sia già stato importato.

Prestare attenzione a maiuscole e minuscole perché il Python® è un linguaggio case sensitive così come i comandi MS.

I valori numerici float vanno scritti sempre con il punto decimale (es. 30.0, non 30). I numeri interi si scrivono senza il punto decimale (es. 30). Le stringhe devono essere racchiuse tra ' (apici) o tra " (virgolette), p.es: 'C8/10' oppure "C8/10".

MS.SetMaterial

Definisce un materiale nella banca locale del progetto e restituisce il suo identificativo numerico (materialID), da usare poi nei comandi MS.Beam e MS.Plate.

Sintassi:

```
materialID = MS.SetMaterial(materialName)
```

Parametri:

Parametro	Tipo	Descrizione
materialName	stringa	Nome del materiale. Valori accettati elencati di seguito.

Valori accettati per materialName:

```
# Calcestruzzo / cemento armato
# 'C8/10','C12/15','C16/20','C20/25','C25/30','C28/35',
# 'C30/37','C32/40','C35/45','C40/50','C45/55','C50/60'
# 'C55/67','C60/75','C70/85','C80/95','C90/105'

# Altri materiali
# 'Acciaio','Legno','Muratura','Alluminio','Ghisa','Vetro'
```

Valore restituito: intero – identificativo del materiale (materialID).

Esempio:

```
matCA = MS.SetMaterial('C25/30')
matAcc = MS.SetMaterial('Acciaio')
matMur = MS.SetMaterial('Muratura')
```

MS.Section

Definisce una sezione trasversale per le travi, identificata da un sectionID progressivo. Esistono due tipologie: profili a caldo UNI in acciaio (sectionType = 1) e sezioni ricorrenti parametriche di qualunque materiale (sectionType = 2).

Nota: non usare MS.Section per le piastre. Lo spessore delle piastre si definisce direttamente in MS.Plate.

Sintassi:

```
# Profilo a caldo UNI (sectionType = 1)
MS.Section(sectionID, 1, sectionName)

# Sezione ricorrente parametrica (sectionType = 2)
MS.Section(sectionID, 2, sectionName, param1, param2, ...)
```

Parametri:

Parametro	Tipo	Descrizione
sectionID	intero	Identificativo univoco della sezione, progressivo da 1.
sectionType	intero	1 = profilo a caldo UNI in acciaio; 2 = sezione ricorrente parametrica.
sectionName	stringa	Per sectionType=1: sigla profilo, es. 'HEA 200', 'IPE 300', 'L 100x10'. Per sectionType=2: 'Rp', 'Cp', 'Rf', 'Cc'.
param1, ...	float	Per sectionType=2: dimensioni geometriche della sezione

Sezioni ricorrenti disponibili (sectionType = 2):

sectionName	Tipo sezione	Descrizione
Rp	Rettangolare piena	param1 = base, param2 = altezza
Cp	Circolare piena	param1 = diametro
Rf	Rettangolare cava	param1 = base, param2 = altezza, param3 = spessore orizzontale, param4 = spessore verticale
Cc	Circolare cava	param1 = diametro, param2 = spessore

Profili a caldo UNI (sectionType = 1):

In section name si mettono le sigle seguite dalla dimensione: HEA, HEB, HEM, HLS, IPE, ILS, IPN, UNP, L. La dimensione è sempre in mm.

Es. 'HEA 200', 'IPE 300'. Per i profili ad L: 'L 100x10' (lati uguali) oppure 'L 100x50x6' (lati diversi).

Valore restituito: nessuno.

Esempio:

```
MS.Section(1, 2, 'Rp', 30.0, 50.0)      # rettangolare piena 30x50 cm
MS.Section(2, 2, 'Cp', 40.0)           # circolare piena diam. 40 cm
MS.Section(3, 2, 'Rf', 20.0, 30.0, 2.0, 2.0) # rettangolare cava
MS.Section(4, 1, 'HEA 200')             # profilo HEA 200
MS.Section(5, 1, 'IPE 300')             # profilo IPE 300
MS.Section(6, 1, 'L 100x10')            # angolare L 100x10
```

MS.Node

Definisce un nodo nello spazio tridimensionale. Il nodeID puo' essere assegnato manualmente (da 1 in poi) oppure lasciato a MasterSap (mettendo nodeID = 0); in quest'ultimo caso il valore restituito va salvato in una variabile per poterlo riutilizzare negli altri comandi.

Nota: se si usa nodeID = 0 per un nodo, usare 0 per tutti i nodi dello script e non mischiare i due approcci.

Sintassi:

```
nodeID_ret = MS.Node(nodeID, coordinataX, coordinataY, coordinataZ)
```

Parametri:

Parametro	Tipo	Descrizione
nodeID	<i>intero</i>	0 = MasterSap assegna automaticamente il nodo. Altrimenti: intero progressivo da 1, univoco nel progetto.
coordinataX	<i>float</i>	Coordinata X del nodo nell'unità di misura corrente.
coordinataY	<i>float</i>	Coordinata Y del nodo nell'unità di misura corrente.
coordinataZ	<i>float</i>	Coordinata Z del nodo nell'unità di misura corrente.

Valore restituito: intero – l'identificativo del nodo (generato da MasterSap se nodeID = 0, altrimenti uguale al nodeID passato nei parametri).

Esempio con nodeID automatico:

```
n1 = MS.Node(0, 0.0, 0.0, 0.0) # origine  
n2 = MS.Node(0, 500.0, 0.0, 0.0)  
n3 = MS.Node(0, 500.0, 500.0, 0.0)  
n4 = MS.Node(0, 0.0, 500.0, 0.0)  
n5 = MS.Node(0, 0.0, 0.0, 300.0) # piano superiore
```

Esempio con nodeID manuale:

```
MS.Node(1, 0.0, 0.0, 0.0)  
MS.Node(2, 500.0, 0.0, 0.0)  
MS.Node(3, 500.0, 500.0, 0.0)
```

MS.DefineGroup

Definisce un nuovo gruppo di elementi. I gruppi TRAVE e PIASTRA hanno sequenze di numerazione indipendenti, entrambe a partire da 1 senza buchi. Un gruppo può contenere al massimo 1999 elementi.

Sintassi:

```
MS.DefineGroup(groupId, groupType, groupName)
```

Parametri:

Parametro	Tipo	Descrizione
groupId	<i>intero</i>	Identificativo del gruppo. Progressivo da 1, indipendente per ciascun groupType. Non sono ammessi buchi di numerazione.
groupType	<i>stringa</i>	'TRAVE' per gruppi di travi/colonne; 'PIASTRA' per gruppi di piastre o macro.
groupName	<i>stringa</i>	Nome descrittivo del gruppo (es. 'Travi primo piano', 'Colonne piano terra', 'Piastrre di platea').

Valore restituito: nessuno.

Esempio:

```
# Gruppi di travi (numerazione indipendente da 1)
MS.DefineGroup(1, 'TRAVE', 'Colonne piano terra')
MS.DefineGroup(2, 'TRAVE', 'Travi piano primo')

# Gruppi di piastre (numerazione separata, riparte da 1)
MS.DefineGroup(1, 'PIASTRA', 'Pareti piano terra')
MS.DefineGroup(2, 'PIASTRA', 'Platea di fondazione')
```


MS.ActivateGroup

Attiva il gruppo specificato. Tutti gli elementi creati dopo questa chiamata (tramite `MS.Beam` o `MS.Plate`) verranno aggiunti al gruppo attivo, fino a quando non viene attivato un nuovo gruppo.

Sintassi:

```
MS.ActivateGroup(groupId, groupType)
```

Parametri:

Parametro	Tipo	Descrizione
groupId	<i>intero</i>	Identificativo del gruppo da attivare.
groupType	<i>stringa</i>	'TRAVE' o 'PIASTRA'. Parametro opzionale: se omissso si assume 'TRAVE'.

Valore restituito: nessuno.

Esempio:

```
MS.ActivateGroup(1, 'TRAVE')
MS.Beam(0, matID, 1, n1, n5)    # va nel gruppo 1 TRAVE
MS.Beam(0, matID, 1, n2, n6)    # va nel gruppo 1 TRAVE

MS.ActivateGroup(2, 'TRAVE')
MS.Beam(0, matID, 2, n5, n6)    # va nel gruppo 2 TRAVE

MS.ActivateGroup(1, 'PIASTRA')
MS.Plate(0, matID, 20.0, n1, n2, n3, n4)  # va nel gruppo 1 PIASTRA
```

MS.Beam

Genera una trave che collega due nodi. L'elemento richiede un materiale, una sezione e deve essere creato con un gruppo attivo.

Sintassi:

```
beamID_ret = MS.Beam(beamID, materialID, sectionID, nodeI, nodeJ)
```

Parametri:

Parametro	Tipo	Descrizione
beamID	<i>intero</i>	0 = MasterSap assegna automaticamente (consigliato). Altrimenti: intero da 1, univoco tra tutti gli elementi del progetto (travi e piastre).
materialID	<i>intero</i>	Identificativo del materiale, ottenuto da MS.SetMaterial().
sectionID	<i>intero</i>	Identificativo della sezione, definita con MS.Section().
nodeI	<i>intero</i>	Identificativo del nodo iniziale (estremo I).
nodeJ	<i>intero</i>	Identificativo del nodo finale (estremo J).

Valore restituito: intero – identificativo dell'elemento generato.

Esempio – telaio con colonne e trave:

```
matID = MS.SetMaterial('C25/30')
MS.Section(1, 2, 'Rp', 30.0, 30.0) # colonne 30x30 cm
MS.Section(2, 2, 'Rp', 30.0, 50.0) # travi 30x50 cm

n1 = MS.Node(0, 0.0, 0.0, 0.0)
n2 = MS.Node(0, 400.0, 0.0, 0.0)
n3 = MS.Node(0, 0.0, 0.0, 300.0)
n4 = MS.Node(0, 400.0, 0.0, 300.0)

MS.DefineGroup(1, 'TRAVE', 'Colonne')
MS.ActivateGroup(1, 'TRAVE')
MS.Beam(0, matID, 1, n1, n3) # colonna sinistra
MS.Beam(0, matID, 1, n2, n4) # colonna destra

MS.DefineGroup(2, 'TRAVE', 'Travi')
MS.ActivateGroup(2, 'TRAVE')
MS.Beam(0, matID, 2, n3, n4) # trave orizzontale
```

MS.Plate

Genera una piastra triangolare o quadrilatero planare. Si usa per modellare pareti, platee, cupole e superfici curve. Piastre adiacenti devono condividere i nodi sul bordo comune. La normale alla piastra segue la regola della mano destra nell'ordine nodeI -> nodeJ -> nodeK (-> nodeL).

Sintassi:

```
# Quadrilatero (4 nodi)
plateID_ret = MS.Plate(plateID, materialID, spess, nodeI, nodeJ, nodeK, nodeL)

# Triangolo (3 nodi)
plateID_ret = MS.Plate(plateID, materialID, spess, nodeI, nodeJ, nodeK)
```

Parametri:

Parametro	Tipo	Descrizione
plateID	<i>intero</i>	0 = MasterSap assegna automaticamente (consigliato). Altrimenti: intero da 1, univoco tra tutti gli elementi del progetto.
materialID	<i>intero</i>	Identificativo del materiale, ottenuto da MS.SetMaterial.
spess	<i>float</i>	Spessore della piastra nell'unita' di misura corrente (minimo 1 cm).
nodeI	<i>intero</i>	Primo nodo.
nodeJ	<i>intero</i>	Secondo nodo.
nodeK	<i>intero</i>	Terzo nodo.
nodeL	<i>intero</i>	Quarto nodo (solo per quadrilateri, opzionale).

Valore restituito: intero – identificativo della piastra generata.

Esempio – parete rettangolare 400x300 cm, spessore 20 cm:

```
matID = MS.SetMaterial('C25/30')
p1 = MS.Node(0, 0.0, 0.0, 0.0)
p2 = MS.Node(0, 400.0, 0.0, 0.0)
p3 = MS.Node(0, 400.0, 0.0, 300.0)
p4 = MS.Node(0, 0.0, 0.0, 300.0)
MS.DefineGroup(1, 'PIASTRA', 'Parete')
MS.ActivateGroup(1, 'PIASTRA')
MS.Plate(0, matID, 20.0, p1, p2, p3, p4)
```

Esempio – parete suddivisa in mesh 2x2:

```
n = {}
for iz, z in enumerate([0.0, 150.0, 300.0]):
    for ix, x in enumerate([0.0, 200.0, 400.0]):
        n[(ix, iz)] = MS.Node(0, x, 0.0, z)

MS.DefineGroup(1, 'PIASTRA', 'Parete mesh')
MS.ActivateGroup(1, 'PIASTRA')
for iz in range(2):
    for ix in range(2):
        MS.Plate(0, matID, 20.0,
                n[(ix, iz)],
                n[(ix+1, iz)],
                n[(ix+1, iz+1)],
                n[(ix, iz+1)])
```

MS.Macro

Genera una macro, ovvero un elemento bidimensionale di grandi dimensioni che MasterSap suddivide internamente in piastre più piccole. Si usa principalmente per platee o per coprire superfici piane ampie (lati tipicamente superiori a 100 cm). A differenza delle piastre, la macro non viene mai suddivisa dallo script.

Importante: ogni macro deve essere inserita in un gruppo PIASTRA dedicato esclusivamente a lei (senza altri elementi).

Sintassi:

```
# Quadrilatero (4 nodi)
MS.Macro(macroName, materialID, spess, nodeI, nodeJ, nodeK, nodeL)
```

```
# Triangolo (3 nodi)
MS.Macro(macroName, materialID, spess, nodeI, nodeJ, nodeK)
```

Parametri:

Parametro	Tipo	Descrizione
macroName	stringa	Nome della macro (max 50 caratteri ASCII). Es. 'Platea fondazione', 'Solaio piano terra'.
materialID	intero	Identificativo del materiale, ottenuto da MS.SetMaterial.
spess	float	Spessore della macro nell'unita' di misura corrente.
nodeI..L	intero	Identificativi dei nodi ai vertici (3 per triangolo, 4 per quadrilatero).

Valore restituito: nessuno.

Esempio – platea di fondazione 10x8 m, spessore 60 cm (misure in cm):

```
matID = MS.SetMaterial('C25/30')

p1 = MS.Node(0, 0.0, 0.0, 0.0)
p2 = MS.Node(0, 1000.0, 0.0, 0.0)
p3 = MS.Node(0, 1000.0, 800.0, 0.0)
p4 = MS.Node(0, 0.0, 800.0, 0.0)

# Gruppo dedicato esclusivamente alla macro
MS.DefineGroup(1, 'PIASTRA', 'Macro platea')
MS.ActivateGroup(1, 'PIASTRA')
MS.Macro('Platea fondazione', matID, 60.0, p1, p2, p3, p4)
```

MS.SetUndo

Fissa un punto di ripristino **prima** della generazione del modello. In caso di necessità di ripartire, il programma potrà tornare a questo stato. Va inserito una sola volta all'inizio dello script.

Sintassi:

```
MS.SetUndo()
```

Parametri: nessuno.

Valore restituito: nessuno.

Esempio:

```
MS.SetUndo()  
# ... generazione del modello ...
```

MS.Redraw

Ridisegna la scena in MasterSap, aggiornando la visualizzazione 3D del modello. Va chiamato una sola volta come ultima istruzione dello script.

Sintassi:

```
MS.Redraw()
```

Parametri: nessuno.

Valore restituito: nessuno.

Esempio:

```
# ... generazione nodi, sezioni, travi ...  
MS.Redraw()    # ultima riga dello script
```

Avviamento di uno script MasterSap nella console MasterPy

Si ricorda che per avviare con successo uno script MasterSap (o per inviare manualmente un comando MS), il programma MasterSap deve essere in esecuzione con un progetto vuoto aperto.

go

Comando semplificato per avviare uno script MasterSap.

Sintassi:

`go (nomefile)`

Parametri:

Parametro	Tipo	Descrizione
nomefile	<i>stringa</i>	nome dello script con estensione .py da eseguire. Questo script si deve trovare nella cartella Documenti dell'utente o nella cartella Download di default del sistema Windows

Esempio:

```
go('myscript.py') #solo con myscript.py nella cartella Documenti dell'utente
```

Note:

Questo comando specifico di MasterPy è stato inserito per semplificare l'avvio di uno script che si trovi nella cartella Documenti dell'utente o nella cartella Download di default del sistema Windows. L'avvio di uno script "myscript.py" si può sempre ottenere tramite il comando standard di Python, specificando un percorso generico:

```
exec(open("C:/Users/utente/Documents/myscript.py").read())
```

Se il file "myscript.py" viene messo nella cartella documenti dell'utente, in modo più semplificato, grazie alla console di MasterPy, si può fare:

```
go('myscript.py')
```

Esempio 1: edificio a due piani

Lo script seguente genera un edificio a due piani con quattro colonne, travi perimetrali e macro di solaio. L'unità di misura è il centimetro. La pianta è 500x500 cm, i piani sono alti 300 cm ciascuno.

```
# =====
# MasterSap Script
# Titolo: Esempio01
# Autore: AG - AMV
# Licenza: Apache-2.0
# Testato su: MasterSap 2026 R1
# Unità: <kg; cm>
# Scopo: Esempio applicativo
# Input: no
# Output: Edificio a 2 piani in c.a.
# Limitazioni: Esempio d'uso
# =====

import MS
MS.SetUndo()

# Materiali
matCA = MS.SetMaterial('C25/30')

# Sezioni
MS.Section(1, 2, 'Rp', 40.0, 40.0) # colonne 40x40 cm
MS.Section(2, 2, 'Rp', 30.0, 60.0) # travi 30x60 cm

# Nodi a terra (Z=0)
n1 = MS.Node(0, 0.0, 0.0, 0.0)
n2 = MS.Node(0, 500.0, 0.0, 0.0)
n3 = MS.Node(0, 500.0, 500.0, 0.0)
n4 = MS.Node(0, 0.0, 500.0, 0.0)
# Nodi piano 1 (Z=300)
n5 = MS.Node(0, 0.0, 0.0, 300.0)
n6 = MS.Node(0, 500.0, 0.0, 300.0)
n7 = MS.Node(0, 500.0, 500.0, 300.0)
n8 = MS.Node(0, 0.0, 500.0, 300.0)
# Nodi piano 2 (Z=600)
n9 = MS.Node(0, 0.0, 0.0, 600.0)
n10 = MS.Node(0, 500.0, 0.0, 600.0)
n11 = MS.Node(0, 500.0, 500.0, 600.0)
n12 = MS.Node(0, 0.0, 500.0, 600.0)

# Colonne piano terra
MS.DefineGroup(1, 'TRAVE', 'Colonne piano terra')
MS.ActivateGroup(1, 'TRAVE')
for base, top in [(n1,n5),(n2,n6),(n3,n7),(n4,n8)]:
    MS.Beam(0, matCA, 1, base, top)
# Colonne piano primo
MS.DefineGroup(2, 'TRAVE', 'Colonne piano primo')
MS.ActivateGroup(2, 'TRAVE')
for base, top in [(n5,n9),(n6,n10),(n7,n11),(n8,n12)]:
    MS.Beam(0, matCA, 1, base, top)
# Travi piano primo
MS.DefineGroup(3, 'TRAVE', 'Travi piano primo')
MS.ActivateGroup(3, 'TRAVE')
for ni, nj in [(n5,n6),(n6,n7),(n7,n8),(n8,n5)]:
    MS.Beam(0, matCA, 2, ni, nj)
# Travi piano secondo
MS.DefineGroup(4, 'TRAVE', 'Travi piano secondo')
MS.ActivateGroup(4, 'TRAVE')
```

```
for ni, nj in [(n9,n10),(n10,n11),(n11,n12),(n12,n9)]:
    MS.Beam(0, matCA, 2, ni, nj)
# Solaio piano primo
MS.DefineGroup(1, 'PIASTRA', 'Solaio piano primo')
MS.ActivateGroup(1, 'PIASTRA')
MS.Macro('Solaio1', matCA, 20.0, n5, n6, n7, n8)
# Solaio piano secondo
MS.DefineGroup(2, 'PIASTRA', 'Solaio piano secondo')
MS.ActivateGroup(2, 'PIASTRA')
MS.Macro('Solaio2', matCA, 20.0, n9, n10, n11, n12)

MS.Redraw()
```

Esempio 2: edificio multipiano

Lo script seguente genera un edificio a tre piani, due piani in c.a. e il piano superiore in acciaio.

```
# =====
# MasterSap Script
# Titolo: Esempio02
# Autore: AG NP - AMV
# Licenza: Apache-2.0
# Testato su: MasterSap 2026 R1
# Unità: <kg; cm>
# Scopo: Esempio applicativo
# Input: no
# Output: Edificio multipiano c.a. e acciaio
# Limitazioni: Esempio d'uso
# =====

import MS
MS.SetUndo()

# Materiali\
mat_c4050 = MS.SetMaterial("C40/50")
mat_c7085 = MS.SetMaterial("C70/85")
mat_acciaio = MS.SetMaterial("Acciaio")

# Sezioni
MS.Section(1, 2, "Rp", 50.0, 50.0)
MS.Section(2, 2, "Rp", 30.0, 50.0)
MS.Section(3, 1, "HEA 300")

# Gruppi
MS.DefineGroup(1, "PIASTRA", "Piastrine di platea")
MS.DefineGroup(1, "TRAVE", "Colonne piano 1 in calcestruzzo")
MS.DefineGroup(2, "TRAVE", "Travi piano 1 in calcestruzzo")
MS.DefineGroup(3, "TRAVE", "Colonne piano 2 in calcestruzzo")
MS.DefineGroup(4, "TRAVE", "Travi piano 2 in calcestruzzo")
MS.DefineGroup(5, "TRAVE", "Colonne piano 3 in acciaio")
MS.DefineGroup(6, "TRAVE", "Travi piano 3 in acciaio")

node_id = 1
beam_id = 1
plate_id = 1
nodes = {}

def get_node(x, y, z):
    global node_id
    key = (float(x), float(y), float(z))
```



```
if key not in nodes:
    nodes[key] = node_id
    MS.Node(node_id, float(x), float(y), float(z))
    node_id += 1
return nodes[key]

# Coordinate in cm
x_cols = [0.0, 260.0, 520.0, 780.0, 1040.0, 1300.0]
y_cols = [0.0, 200.0, 400.0, 600.0, 800.0]
z_levels = [0.0, 350.0, 700.0, 1050.0]

# Platea almeno 1 m oltre la pianta in ogni direzione.
# La maglia delle piastre e costruita in modo che i nodi di base dei pilastri
# coincidano con nodi delle piastre e che ogni piastra abbia lato <= 70 cm.
x_plate = [
    -100.0, -50.0, 0.0,
    65.0, 130.0, 195.0, 260.0,
    325.0, 390.0, 455.0, 520.0,
    585.0, 650.0, 715.0, 780.0,
    845.0, 910.0, 975.0, 1040.0,
    1105.0, 1170.0, 1235.0, 1300.0,
    1350.0, 1400.0,
]
y_plate = [
    -100.0, -50.0, 0.0, 50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0,
    400.0, 450.0, 500.0, 550.0, 600.0, 650.0, 700.0, 750.0, 800.0, 850.0,
    900.0,
]

# Nodi della maglia strutturale ai diversi livelli.
for z in z_levels:
    for x in x_cols:
        for y in y_cols:
            get_node(x, y, z)

# Nodi della platea sul piano z = 0.
for x in x_plate:
    for y in y_plate:
        get_node(x, y, 0.0)

# Platea in piastre di spessore 50 cm.
MS.ActivateGroup(1)
for ix in range(len(x_plate) - 1):
    for iy in range(len(y_plate) - 1):
        n1 = get_node(x_plate[ix], y_plate[iy], 0.0)
        n2 = get_node(x_plate[ix + 1], y_plate[iy], 0.0)
        n3 = get_node(x_plate[ix + 1], y_plate[iy + 1], 0.0)
        n4 = get_node(x_plate[ix], y_plate[iy + 1], 0.0)
        MS.Plate(plate_id, mat_c7085, 50.0, n1, n2, n3, n4)
        plate_id += 1

# Colonne piano 1 in calcestruzzo: da z = 0 a z = 350 cm.
MS.ActivateGroup(1)
for x in x_cols:
    for y in y_cols:
        n1 = get_node(x, y, 0.0)
        n2 = get_node(x, y, 350.0)
        MS.Beam(beam_id, mat_c4050, 1, n1, n2)
        beam_id += 1

# Travi piano 1 in calcestruzzo al livello z = 350 cm.
```

```
MS.ActivateGroup(2)
for y in y_cols:
    for ix in range(len(x_cols) - 1):
        n1 = get_node(x_cols[ix], y, 350.0)
        n2 = get_node(x_cols[ix + 1], y, 350.0)
        MS.Beam(beam_id, mat_c4050, 2, n1, n2)
        beam_id += 1
for x in x_cols:
    for iy in range(len(y_cols) - 1):
        n1 = get_node(x, y_cols[iy], 350.0)
        n2 = get_node(x, y_cols[iy + 1], 350.0)
        MS.Beam(beam_id, mat_c4050, 2, n1, n2)
        beam_id += 1

# Colonne piano 2 in calcestruzzo: da z = 350 a z = 700 cm.
MS.ActivateGroup(3)
for x in x_cols:
    for y in y_cols:
        n1 = get_node(x, y, 350.0)
        n2 = get_node(x, y, 700.0)
        MS.Beam(beam_id, mat_c4050, 1, n1, n2)
        beam_id += 1

# Travi piano 2 in calcestruzzo al livello z = 700 cm.
MS.ActivateGroup(4)
for y in y_cols:
    for ix in range(len(x_cols) - 1):
        n1 = get_node(x_cols[ix], y, 700.0)
        n2 = get_node(x_cols[ix + 1], y, 700.0)
        MS.Beam(beam_id, mat_c4050, 2, n1, n2)
        beam_id += 1
for x in x_cols:
    for iy in range(len(y_cols) - 1):
        n1 = get_node(x, y_cols[iy], 700.0)
        n2 = get_node(x, y_cols[iy + 1], 700.0)
        MS.Beam(beam_id, mat_c4050, 2, n1, n2)
        beam_id += 1

# Colonne piano 3 in acciaio HEA: da z = 700 a z = 1050 cm.
MS.ActivateGroup(5)
for x in x_cols:
    for y in y_cols:
        n1 = get_node(x, y, 700.0)
        n2 = get_node(x, y, 1050.0)
        MS.Beam(beam_id, mat_acciaio, 3, n1, n2)
        beam_id += 1

# Travi piano 3 in acciaio HEA al livello z = 1050 cm.
MS.ActivateGroup(6)
for y in y_cols:
    for ix in range(len(x_cols) - 1):
        n1 = get_node(x_cols[ix], y, 1050.0)
        n2 = get_node(x_cols[ix + 1], y, 1050.0)
        MS.Beam(beam_id, mat_acciaio, 3, n1, n2)
        beam_id += 1
for x in x_cols:
    for iy in range(len(y_cols) - 1):
        n1 = get_node(x, y_cols[iy], 1050.0)
        n2 = get_node(x, y_cols[iy + 1], 1050.0)
        MS.Beam(beam_id, mat_acciaio, 3, n1, n2)
        beam_id += 1
MS.Redraw()
```

Esempio 3: volta a crociera (solo nodi)

Lo script seguente genera una volta a crociera a tutto sesto su impianto quadrato (solo nodi).

```
# =====
# MasterSap Script
# Titolo: Esempio03
# Autore: AG - AMV
# Licenza: Apache-2.0
# Testato su: MasterSap 2026 R1
# Unità: <kg; cm>
# Scopo: Esempio applicativo
# Input: Raggio, lato, passo
# Output: Volta a crociera su impianto quadrato (solo nodi)
# Limitazioni: Esempio d'uso
# =====

import math
import MS
MS.SetUndo()

R = 500.0
LATO = 1500.0
PASSO_ANG = 10.0

# Volta a crociera su impianto quadrato LATO x LATO.
# Si generano i nodi dei 4 archi di imposta a tutto sesto
# e i nodi di intersezione tra:
# - le rette che uniscono punti corrispondenti degli archi nord-sud
# - le rette che uniscono punti corrispondenti degli archi est-ovest

node_id = 1
node_map = {}

def add_node(x, y, z):
    global node_id
    key = (round(x, 6), round(y, 6), round(z, 6))
    if key not in node_map:
        MS.Node(node_id, float(x), float(y), float(z))
        node_map[key] = node_id
        node_id += 1
    return node_map[key]

def rounded(value):
    return round(value, 6)

# Archi est-ovest: lati AB e CD
# Archi nord-sud: lati BC e DA
arc_ab = []
arc_bc = []
arc_cd = []
arc_da = []

# Arco lato AB: piano y = -LATO/2, sviluppo su x
ang = 180.0
while ang >= 0.0:
    rad = math.radians(ang)
    x = R * math.cos(rad)
    y = -LATO / 2.0
    z = R * math.sin(rad)
    add_node(x, y, z)
    arc_ab.append((x, y, z))
    ang -= PASSO_ANG
```

```
# Arco lato BC: piano x = LATO/2, sviluppo su y
ang = 180.0
while ang >= 0.0:
    rad = math.radians(ang)
    x = LATO / 2.0
    y = R * math.cos(rad)
    z = R * math.sin(rad)
    add_node(x, y, z)
    arc_bc.append((x, y, z))
    ang -= PASSO_ANG

# Arco lato CD: piano y = LATO/2, sviluppo su x
ang = 180.0
while ang >= 0.0:
    rad = math.radians(ang)
    x = -R * math.cos(rad)
    y = LATO / 2.0
    z = R * math.sin(rad)
    add_node(x, y, z)
    arc_cd.append((x, y, z))
    ang -= PASSO_ANG

# Arco lato DA: piano x = -LATO/2, sviluppo su y
ang = 180.0
while ang >= 0.0:
    rad = math.radians(ang)
    x = -LATO / 2.0
    y = -R * math.cos(rad)
    z = R * math.sin(rad)
    add_node(x, y, z)
    arc_da.append((x, y, z))
    ang -= PASSO_ANG

# Per ogni quota z si raccolgono:
# - i valori x dei punti corrispondenti sugli archi est-ovest AB/CD
# - i valori y dei punti corrispondenti sugli archi nord-sud BC/DA
# Ogni intersezione tra queste due famiglie di rette genera un nuovo nodo.
xs_by_z = {}
ys_by_z = {}

for x, y, z in arc_ab:
    z_key = rounded(z)
    if z_key not in xs_by_z:
        xs_by_z[z_key] = set()
    xs_by_z[z_key].add(rounded(x))

for x, y, z in arc_cd:
    z_key = rounded(z)
    if z_key not in xs_by_z:
        xs_by_z[z_key] = set()
    xs_by_z[z_key].add(rounded(x))

for x, y, z in arc_bc:
    z_key = rounded(z)
    if z_key not in ys_by_z:
        ys_by_z[z_key] = set()
    ys_by_z[z_key].add(rounded(y))

for x, y, z in arc_da:
    z_key = rounded(z)
    if z_key not in ys_by_z:
        ys_by_z[z_key] = set()
    ys_by_z[z_key].add(rounded(y))

for z_key in sorted(xs_by_z.keys()):
    if z_key not in ys_by_z:
```

```
        continue
    xs = sorted(xs_by_z[z_key])
    ys = sorted(ys_by_z[z_key])
    for x_val in xs:
        for y_val in ys:
            add_node(x_val, y_val, z_key)
```

```
MS.Redraw()
```

Esempio 4: travatura reticolare Mohnie

Lo script seguente genera una volta a crociera a tutto sesto su impianto quadrato (solo nodi).

```
# =====
# MasterSap Script
# Titolo: Esempio04
# Autore: AG - AMV
# Licenza: Apache-2.0
# Testato su: MasterSap 2026 R1
# Unità: <kg; m>
# Scopo: Esempio applicativo
# Input: Total span, half span, roof angle, initial height
# Output: Steel Mohnie truss with alternating diagonals
# Limitazioni: Esempio d'uso
# =====

import math
MS.SetUndo()

# -----
# Parameters
# -----
total_span = 20.0
half_span = total_span / 2.0
roof_angle_deg = 10.0
roof_angle_rad = math.radians(roof_angle_deg)
initial_height = 1.0

# Six verticals for each roof slope, counting the ridge on both slopes.
# Unique x positions are therefore: 0, 2, 4, ..., 20
panel_x = [0.0, 2.0, 4.0, 6.0, 8.0, 10.0, 12.0, 14.0, 16.0, 18.0, 20.0]

def top_z(x):
    if x <= half_span:
        return initial_height + math.tan(roof_angle_rad) * x
    return initial_height + math.tan(roof_angle_rad) * (total_span - x)

def fmt(v):
    return float(f"{v:.6f}")

# -----
# Material and sections
# -----
material_id = MS.SetMaterial("Acciaio")

# sectionID, sectionType, sectionName
MS.Section(1, 1, "L 110x10")
MS.Section(2, 1, "L 90x10")
MS.Section(3, 1, "L 80x8")
MS.Section(4, 1, "L 60x6")
```

```

section_top = 1
section_bottom = 2
section_vertical = 3
section_diagonal = 4

# -----
# Groups
# -----
MS.DefineGroup(1, "TRAVE", "Travatura reticolare Mohnie")
MS.ActivateGroup(1)

# -----
# Nodes
# Bottom chord lies on Z = 0.0, Y = 0.0
# Top chord follows the two roof slopes
# -----
bottom_nodes = []
top_nodes = []

node_id = 1

for x in panel_x:
    MS.Node(node_id, fmt(x), 0.0, 0.0)
    bottom_nodes.append(node_id)
    node_id += 1

for x in panel_x:
    MS.Node(node_id, fmt(x), 0.0, fmt(top_z(x)))
    top_nodes.append(node_id)
    node_id += 1

# -----
# Beams
# -----
beam_id = 1

def add_beam(section_id, node_i, node_j):
    global beam_id
    MS.Beam(beam_id, material_id, section_id, node_i, node_j)
    beam_id += 1

# Bottom chord
for i in range(len(bottom_nodes) - 1):
    add_beam(section_bottom, bottom_nodes[i], bottom_nodes[i + 1])

# Top chord
for i in range(len(top_nodes) - 1):
    add_beam(section_top, top_nodes[i], top_nodes[i + 1])

# Verticals
for i in range(len(panel_x)):
    add_beam(section_vertical, bottom_nodes[i], top_nodes[i])

# Alternating diagonals, one for each panel
# Even panel: from lower left to upper right
# Odd panel: from upper left to lower right
for i in range(len(panel_x) - 1):
    if i % 2 == 0:
        add_beam(section_diagonal, bottom_nodes[i], top_nodes[i + 1])
    else:
        add_beam(section_diagonal, top_nodes[i], bottom_nodes[i + 1])

MS.Redraw()

```